

03_pa_b

Experimentation

Q26. How do you turn a vague product hunch into a testable experiment hypothesis?

Ideal answer. A good hypothesis is falsifiable and pre-commits to a direction, a mechanism, and a magnitude: "Because [behavioral insight], if we [change], then [primary metric] will move by [at least the MDE], measured over [window], without harming [guardrail]." I force the mechanism clause because it prevents "we changed the button color and conversion went up" stories that don't generalize — the mechanism is what you actually learn. I also pre-register the primary metric, the analysis unit, the segment cuts, and the decision rule before I see data, so I can't retrofit a winner. The single most common failure I avoid is the unfalsifiable hypothesis ("this will improve engagement") that no result could ever contradict. Concretely, a Helios-style hypothesis reads: "Because Paid Search add_to_cart→begin_checkout rate dropped 6pp driven by a mobile rate-change (not mix), if we add an express-checkout entry point, begin_checkout rate recovers ≥3pp over 14 days, with bounce rate as a guardrail." **Why Helios demonstrates it.** The Prescribe agent emits experiment cards whose hypothesis is *derived* from the decompose_change rate-vs-mix verdict, so the mechanism clause is grounded in a quantified, SQL-verified diagnosis rather than a guess. **Follow-ups.** How do you decide the experiment's primary metric vs secondary metrics? / What stops a stakeholder from changing the success metric after seeing results? / When is a directional hypothesis ("X goes up") insufficient and you need a magnitude?

Q27. Walk me through sizing an A/B test: MDE, power, significance, and sample size.

Ideal answer. Sample size is driven by four levers: baseline rate p , the minimum detectable effect (MDE), significance α (false-positive tolerance), and power $1-\beta$ (false-negative tolerance, usually 80%). For a two-proportion test the per-arm $n \approx (z_{1-\alpha/2} + z_{1-\beta})^2 \cdot [p_0(1-p_0) + p_1(1-p_1)] / (p_1 - p_0)^2$. The non-negotiable discipline is to set the MDE from the *smallest business-meaningful lift*, not from "what's detectable in two weeks" — sizing to a flattering MDE is how teams ship underpowered tests that can't distinguish a real null from noise. I always convert n into a *runtime* using actual daily eligible traffic and then sanity-check against business calendar (holidays, full weekly cycles). If the runtime is implausibly long, that's signal to widen the MDE, pick a more sensitive proxy metric, or not run the test at all. **Why Helios demonstrates it.** experiment-mcp exposes `power_analysis` and `runtime_estimate` exactly so every Prescribe card carries an MDE, required n per arm, and a feasibility-checked runtime in days against the funnel's measured traffic — no card ships "go run an A/B test" without the math attached. **Follow-ups.** How does variance reduction (CUPED) change your sample-size math? / Why is MDE the lever you should

set first, not last? / How do you size a test on a continuous metric like revenue-per-session vs a proportion?

Q28. What are guardrail metrics and why do they matter more than the primary metric sometimes?

Ideal answer. Guardrail metrics are the metrics you're *not* trying to move but refuse to break — latency, error rate, refund rate, unsubscribe rate, overall revenue. They exist because almost any local optimization can shift harm elsewhere: a checkout redesign that lifts `begin_checkout` but tanks payment success is a net loss disguised as a win. I treat guardrails asymmetrically — the primary metric needs to clear a positive MDE, but a guardrail only needs to *not regress* beyond a tolerance, so I run a non-inferiority test on it rather than demanding significance. The maturity point is recognizing that a "winning" experiment with a silently degraded guardrail is the most dangerous outcome, because it ships and the damage compounds. I pre-register guardrails alongside the primary so no one quietly drops the inconvenient one after the fact. **Why Helios demonstrates it.** Every Prescribe experiment card names an explicit guardrail (e.g., bounce rate or downstream purchase rate), and the Critic agent will DOWNGRADE a prescription whose proposed change plausibly trades a near-term step-rate gain for a downstream-revenue loss.

Follow-ups. How do you set the tolerance band on a guardrail? / What's the difference between a guardrail and a secondary metric? / How do you handle a test that wins on primary but trips a guardrail by a hair?

Q29. Explain the peeking problem and how sequential testing addresses it.

Ideal answer. Peeking is checking a fixed-horizon test repeatedly and stopping the moment $p < 0.05$ — this inflates the false-positive rate far above the nominal α because you get many independent chances to cross the threshold (with daily peeks over two weeks, true type-I error can exceed 30%). The naive fix is "don't look," which is operationally unrealistic. The correct fixes are *sequential* methods that are valid under continuous monitoring: group-sequential designs with alpha-spending functions (O'Brien-Fleming, Pocock), or always-valid inference via mixture sequential probability ratio tests / always-valid confidence sequences. These spend your error budget across looks so the *anytime* false-positive rate stays bounded. The tradeoff is a modest sample-size premium versus a fixed-horizon test in exchange for the freedom to stop early and to look whenever you like. **Why Helios demonstrates it.** Helios runs on observational historical data so it doesn't *run* live sequential tests, but experiment-mcp's design templates pre-commit to a fixed horizon and decision rule precisely to model the discipline that prevents peeking — and the offline benchmark's significance gating uses pre-registered thresholds rather than stopping when a number looks good. **Follow-ups.** Why does daily peeking inflate type-I error so much? / O'Brien-Fleming vs always-valid confidence sequences — when would you pick each? / How does peeking interact with the multiple-comparisons problem across many metrics?

Q30. What are novelty and primacy effects, and how do you keep them from fooling you?

Ideal answer. Novelty effect is when users over-engage with anything new simply because it's new, inflating early treatment numbers that decay; primacy (change-aversion) is the mirror image — existing users resist a change and depress early numbers that recover as they adapt. Both make the *first few days* unrepresentative of steady state. I defend against them by running tests for at least one to two full behavioral cycles (commonly 2+ weeks), plotting the treatment effect *over time* rather than trusting a single pooled estimate, and segmenting new vs returning users since novelty/primacy mostly bite the returning cohort. If the effect curve hasn't stabilized, the test isn't done regardless of significance. The honest read is that a large day-1 lift that decays toward zero is usually novelty, not a durable win. **Why Helios demonstrates it.** This is the same logic as Helios's Critic seasonality/transient check: a finding driven by a short-lived spike gets challenged the same way a novelty-driven experiment result would, and the eval suite includes seasonality-decoy scenarios that punish mistaking a transient bump for a durable shift. **Follow-ups.** How long is "long enough" and how do you decide? / How would you formally test whether the treatment effect is stable over time? / Why segment new vs returning users when diagnosing novelty?

Q31. When should you NOT run an A/B test?

Ideal answer. Don't experiment when you can't get adequate power in a reasonable window (low traffic or a rare event makes the required runtime months long); when the change is a one-way door or ethically/legally mandated (you ship it regardless of result); when network/spillover effects violate the stable-unit assumption (marketplace, social, pricing changes leak across arms); when the cost of the experiment exceeds the value of the information; or when the decision is reversible and cheap enough that just shipping and monitoring is faster learning. In those cases I reach for alternatives: quasi-experiments, interrupted time series, holdout geographies, or bandits for low-stakes optimization. The senior move is treating "should we test at all?" as the first question, not "how do we test?" — running an underpowered test you'll over-interpret is worse than running none. **Why Helios demonstrates it.** Helios is built on *observational* historical GA4 data where live A/B testing is impossible, so it designs and *sizes* experiments and then performs quasi-experimental readbacks (pre/post, difference-in-differences) — it explicitly does not claim to run live A/Bs, which is itself a "when not to experiment" judgment baked into the architecture. **Follow-ups.** How do network effects break the A/B assumption, and what do you do instead? / How do you value the information from an experiment to decide if it's worth running? / What's your fallback when traffic is too low to power any test?

Q32. How do you measure impact when you can't randomize — pre/post and difference-in-differences?

Ideal answer. With observational data I lean on quasi-experimental designs that approximate a counterfactual. Naive pre/post is the weakest because it confounds the intervention with

everything else that changed over time (seasonality, macro trends), so I only trust it with a stable baseline and explicit seasonality controls. Difference-in-differences is stronger: I compare the treated unit's before→after change against an *untreated control* unit's same-period change, which differences out any shock common to both — its load-bearing assumption is *parallel trends* (the two would have moved together absent treatment), which I validate on pre-period data. For a single treated series I'll use interrupted time series or a synthetic control (a weighted blend of donor units that reconstructs the pre-period). The honest caveat: these identify effects only under assumptions randomization would have guaranteed for free, so I state those assumptions out loud.

```
-- Difference-in-differences on funnel conversion
SELECT
  AVG(IF(treated AND post, rate, NULL)) - AVG(IF(treated AND NOT post, rate, NULL))
  - (AVG(IF(NOT treated AND post, rate, NULL)) - AVG(IF(NOT treated AND NOT post, rate, NULL)))
  AS did_effect
FROM daily_segment_rates;
```

Why Helios demonstrates it. Because the GA4 dataset is observational (Nov 2020–Jan 2021), Helios's experiment-mcp prescribes the *design* and then reads back impact via pre/post and diff-in-diff rather than asserting causal A/B results — and the Critic enforces the parallel-trends / seasonality caveats that keep a pre/post number from being oversold as causal. **Follow-ups.** How do you validate the parallel-trends assumption before trusting DiD? / When would you choose synthetic control over DiD? / How do you communicate causal uncertainty from a quasi-experiment to an executive?

North Star Metrics

Q33. What makes a good North Star Metric, and how would you choose one for an e-commerce store?

Ideal answer. A North Star Metric (NSM) should sit at the intersection of customer value and durable business value, be a leading-ish indicator of revenue (not revenue itself), be measurable today, and be something teams can actually influence. For an e-commerce store I'd resist "revenue" as the NSM because it's a lagging outcome that every team can claim and none can directly move; I'd prefer something like *weekly purchasing customers* or *number of completed purchases per active user* — value is realized at purchase, and it captures both acquisition and retention. The test I apply: if this number goes up sustainably for a year, is the business healthier? If yes, and teams can map their work to it, it's a candidate. I pair it with 2–4 input metrics so a single number never becomes the only thing the org looks at. **Why Helios demonstrates it.** Helios treats *purchase* as the terminal funnel event and quantifies revenue-at-risk in dollars, so its diagnoses are always anchored to the value-realizing step — the NSM-style outcome — while

decomposing the funnel steps that *lead* to it. **Follow-ups.** Why not just use revenue as the North Star? / How many input metrics should sit under a North Star? / How would you NSM a pre-monetization product where purchases don't exist yet?

Q34. Distinguish leading from lagging indicators with concrete examples.

Ideal answer. Lagging indicators measure outcomes after the fact (revenue, churn, LTV, retention) — accurate but you can't steer by them in time. Leading indicators move *first* and predict the lagging outcome (add_to_cart rate, week-1 activation, checkout-start rate, repeat-visit frequency), so they're what you actually manage day to day. The art is establishing that the "leading" metric is genuinely predictive of the lagging one rather than just correlated noise — I'd validate the lead-lag relationship statistically (e.g., does this week's activation predict next month's retention?) before trusting it. A practical rule: dashboards full of lagging metrics tell you the patient's temperature is high but not why; leading metrics tell you which intervention to make now. I report both, but I *act* on leading and *judge* on lagging. **Why Helios demonstrates it.** Helios's funnel is an explicit leading→lagging chain (session_start → view_item → add_to_cart → ... → purchase); when it diagnoses a purchase shortfall it walks *upstream* to the leading step rate that moved, which is precisely "act on the leading indicator that predicts the lagging outcome." **Follow-ups.** How do you statistically confirm a metric is leading, not coincident? / Give an example where a leading indicator decoupled from its lagging outcome. / How does the lead time of a leading metric affect how you use it?

Q35. What is an input metrics tree and how does it connect daily work to the North Star?

Ideal answer. An input metrics tree (or metric tree) decomposes the North Star into the multiplicative or additive drivers a team can directly move, level by level, until you reach metrics that map to specific initiatives. For purchases you might decompose: Purchases = Sessions × (Purchase rate), and Purchase rate = product of step conversion rates across the funnel; each step then fans out into the levers behind it (traffic mix, page performance, friction). The discipline is that the tree must *reconcile* — children should roll up to the parent arithmetically — so you can attribute a North Star movement to a specific branch. This is what lets a PM owning "checkout friction" see exactly how their step rate flows up to the company NSM. Without the tree, every team optimizes locally and no one can explain why the top number moved. **Why Helios demonstrates it.** Helios's semantic layer plus `decompose_change` is an operational metric tree: an aggregate rate $R = \sum w_i r_i$ is decomposed into mix, rate, and interaction contributions per segment, so a North-Star-level movement is mechanically attributed down the tree to the exact segment and step that caused it. **Follow-ups.** How do you keep a metric tree reconciling when metrics are ratios? / What happens when two branches of the tree move in opposite directions? / How granular should the leaves of the tree be?

Q36. Explain Goodhart's Law and metric gaming — how do you design metrics that resist it?

Ideal answer. Goodhart's Law: "when a measure becomes a target, it ceases to be a good measure" — optimizing a proxy hard enough decouples it from the underlying goal it was standing in for. Classic gaming: a "sessions" target met by splitting one visit into many, or an "add_to_cart" target hit while purchases fall. I design against it with three tactics: pair every target metric with a guardrail or counter-metric that catches the cheap exploit (engagement paired with retention; clicks paired with downstream conversion); prefer metrics closer to true value (purchases over add-to-carts) so the proxy gap is small; and define metrics governed and centrally so a team can't quietly re-spec the numerator to flatter itself. The senior framing is that the more pressure a metric carries, the closer it must sit to real value and the more its definition must be locked down. **Why Helios demonstrates it.** Helios's governed semantic layer (semantic-mcp as the *only* SQL path) means no agent or analyst can redefine "purchase rate" to game a result, and the Critic's adversarial refutation explicitly hunts for the "looks good but is an artifact" failure mode that Goodhart-style gaming produces. **Follow-ups.** How do you choose the counter-metric that catches gaming of a target? / Why does governing a metric's definition reduce gaming? / Give an example where moving a metric closer to true value introduced a new gaming surface.

Q37. How do you avoid a single-metric tunnel vision while still having one North Star?

Ideal answer. The North Star is a *focusing* device, not the only number — the failure mode is treating it as the sole KPI and optimizing it to the detriment of everything around it. I surround it with a small balanced set: the NSM, its input tree, and a ring of guardrails covering the dimensions the NSM ignores (quality, cost, fairness across segments, long-term retention). I also re-examine the NSM periodically, because the right North Star at one growth stage (activation early, retention/monetization later) is wrong at another. The judgment is holding the tension: enough focus that the org rows together, enough breadth that you notice when "winning" on the NSM is quietly losing the business. A North Star with no guardrails is just a Goodhart trap waiting to spring. **Why Helios demonstrates it.** Helios never reports a single number in isolation — its Decision Brief pairs the headline movement with the mix/rate decomposition, revenue-at-risk, *and* the Critic's caveats, so the executive sees the focal metric inside its guardrail context, not alone. **Follow-ups.** When should an NSM change as the company matures? / How big should the guardrail ring be before it's just a dashboard again? / How do you detect that you're optimizing the NSM at the expense of the business?

Q38. How would you validate that your North Star actually predicts long-term business value?

Ideal answer. I'd test the predictive link rather than assume it: build a cohort dataset, measure each cohort's early NSM trajectory, and check whether it explains later lagging outcomes (revenue, retention, LTV) — correlation over time, lead-lag regression, or out-of-sample prediction. If the NSM doesn't predict the outcome it's supposed to lead, it's the wrong star. I'd

also stress it across segments to make sure the relationship isn't a Simpson's-paradox artifact of mix. The honest constraint: predicting *long-term* value needs a long observation window, and with a short dataset I can only validate short-horizon proxies and must state the assumption that the relationship persists. That caveat is part of the answer, not a weakness in it. **Why Helios demonstrates it.** Helios is candid that its ~3-month GA4 window defers true LTV / long-horizon retention, so it reports short-horizon proxies *with stated assumptions* — exactly the disciplined honesty required when you can't yet prove the long-term predictive link of a North Star. **Follow-ups.** What statistical test confirms the NSM predicts the lagging outcome? / How does a short data window limit which North Star you can validate? / How would Simpson's paradox fool you into trusting a bad North Star?

Segmentation

Q39. Explain RFM segmentation and when you'd reach for it.

Ideal answer. RFM scores customers on three behavioral axes: Recency (how recently they purchased), Frequency (how often), and Monetary (how much) — typically by quintile-ranking each axis and combining the scores into segments like "champions," "at-risk," and "hibernating." It's powerful because it's purely behavioral, computable from transaction logs alone, and maps directly to action (champions get loyalty perks, at-risk get win-back campaigns). I reach for it for retention and CRM/lifecycle targeting where the question is "who do I treat differently and how?" rather than prediction. Its limits: it's descriptive not predictive, it ignores *why* behavior changed, and the quintile cutoffs need to be revisited as the base shifts. I'd use it as a fast, actionable first pass before reaching for a model.

```
SELECT user_pseudo_id,  
  NTILE(5) OVER (ORDER BY MAX(event_date)) AS r,  
  NTILE(5) OVER (ORDER BY COUNT(*))        AS f,  
  NTILE(5) OVER (ORDER BY SUM(revenue))     AS m  
FROM purchases GROUP BY user_pseudo_id;
```

Why Helios demonstrates it. stats-mcp exposes `rfm_segment` as a deterministic, governed primitive, so RFM scoring runs through the same audited stats path as decomposition and significance testing rather than being hand-rolled SQL per analysis. **Follow-ups.** Why quintiles and not k-means clustering for RFM? / How do you keep RFM cutoffs stable as the customer base grows? / How would you make RFM predictive rather than just descriptive?

Q40. Behavioral vs demographic segmentation — which do you trust and why?

Ideal answer. Demographic segmentation (age, geo, device) is easy to get and easy to explain but often weakly predictive of behavior — two 25-year-olds in the same city can have opposite

purchase intent. Behavioral segmentation (what users actually *did*: pages viewed, cart actions, purchase cadence, channel) is usually far more predictive and more actionable because it's closer to the moment of value. I default to behavioral for product and lifecycle decisions, and use demographic mainly for acquisition targeting where behavior isn't observable yet. The nuance: demographics are a useful *prior* before behavior accumulates (cold start), and the strongest segmentations combine both. The trap to avoid is treating an easily-measured demographic cut as causal when it's just a proxy for an underlying behavior. **Why Helios demonstrates it.** Helios's 16 governed dimensions blend behavioral cuts (funnel step reached, channel group, session source/medium) with contextual ones (device, geo), and its diagnoses prioritize *behavioral* rate-change segments because that's where real, actionable signal lives — composition/demographic shifts are flagged as mix effects, not behavior. **Follow-ups.** When is a demographic segment actually a proxy for a behavior? / How do you segment a brand-new user before any behavior exists? / How would you test which segmentation scheme is more predictive?

Q41. What makes a segment actionable, and how do you avoid useless segmentation?

Ideal answer. A segment is actionable only if it's (1) measurable from data you have, (2) substantial enough to matter, (3) differentiable — members respond differently to a treatment than non-members, and (4) reachable — you can actually do something different for them. The classic waste is a statistically tidy segment you can neither target nor treat differently (interesting cluster, no lever), or a segment so small the effort exceeds the payoff. My test before shipping a segmentation: name the *specific different action* each segment gets. If "at-risk users" and "champions" would receive the same treatment, the split earned nothing. I'd rather have three crude, actionable segments than twelve elegant, inert ones. **Why Helios demonstrates it.** Helios always ties a segment-level finding to a *prescribed action* — when decompose_change attributes a drop to, say, mobile Paid Search, the Prescribe agent emits an experiment card targeting that exact reachable segment, so segmentation never terminates in an un-actionable cluster. **Follow-ups.** How small is too small for a segment to be worth treating? / How do you test that a segment is differentiable (responds differently)? / What do you do with an interesting-but-unreachable segment?

Q42. What is the "segment of one" / personalization, and what are its tradeoffs?

Ideal answer. "Segment of one" is the limit of segmentation — personalizing to the individual via models that predict each user's next-best-action, propensity, or recommendation, rather than bucketing into coarse groups. The upside is relevance and lift; the downsides are real: sparse per-user data (most users have thin histories), the cold-start problem, infrastructure and latency cost, harder explainability and governance, and filter-bubble/fairness risks. I treat it as a spectrum, not a binary — most value is captured by good behavioral *cohorts*, and true 1:1 is worth it only where data density and stakes justify the complexity. The mature stance is starting with actionable

segments and graduating to personalization only where it measurably beats the cohort approach.

Why Helios demonstrates it. Helios deliberately operates at the cookie-grain segment level (`user_pseudo_id`, since `user_id` is almost always null), which is honest about *why* fine-grained per-person personalization isn't credible on this dataset — identity is a cookie-based approximation, so segment-level diagnosis is the defensible altitude. **Follow-ups.** When does a segment-of-one beat well-chosen cohorts enough to justify the cost? / How does sparse per-user data limit personalization? / How would cookie-grain identity (no stable `user_id`) constrain a personalization system?

Q43. How do you decide segmentation granularity without slicing into noise?

Ideal answer. Granularity is a bias-variance tradeoff: coarse segments hide real heterogeneity (Simpson's paradox lurks), fine segments fragment into samples too small for stable estimates. I let *sample size and effect stability* set the floor — a segment needs enough volume that its rate has a tight enough confidence interval to act on — and I let *actionability* set the useful ceiling (don't split finer than you can treat). Practically I'll start coarse, decompose, and only drill into a sub-segment when its contribution is both large and statistically distinguishable from noise. The discipline that prevents over-slicing is requiring significance before trusting a small-segment delta; otherwise you're chasing sampling variance and calling it insight. **Why Helios**

demonstrates it. Helios's Diagnose agent does best-first hypothesis-tree RCA — it drills *rate effects before mix effects* and only deeper into segments that materially move — while the Critic explicitly refutes findings built on *insufficient sample*, which is the over-granular-segmentation failure mode. **Follow-ups.** What's your rule for the minimum sample a segment needs before you trust its rate? / How does Simpson's paradox push you toward finer segmentation? / How do you stop a drill-down from becoming a multiple-comparisons fishing trip?

Q44. A segment looks like it's driving a metric change — how do you confirm it's real and not a mix artifact?

Ideal answer. First I separate two questions: did this segment's *behavior* change (rate), or did its *share of traffic* change (mix)? A naive "largest absolute segment delta" reading conflates them and routinely fingers the wrong cause. I run the mix/rate/interaction decomposition: $\Delta R = \sum (dw_i \cdot r_i)$ [mix] + $\sum (w_i \cdot dr_i)$ [rate] + $\sum (dw_i \cdot dr_i)$ [interaction]; if the segment's contribution is mostly mix, the "driver" is just composition shifting, not behavior. Then I test the rate change for significance so I'm not chasing a small-sample wobble, and I check for confounds (seasonality, a launch, a tracking change). Only a contribution that is rate-driven, significant, and confound-free earns a causal-sounding story. This is exactly how you dissolve Simpson's paradox in attribution. **Why Helios demonstrates it.** This is Helios's technical centerpiece: `decompose_change` deterministically splits a movement into mix/rate/interaction, the Diagnose agent prioritizes the rate-driven (real-behavior) segments, and the Critic adversarially challenges any finding as a

possible mix-confound — beating a naive baseline 85% vs ~45% on the labeled benchmark.

Follow-ups. Walk me through the mix vs rate decomposition arithmetic. / How does a pure mix shift create a Simpson's-paradox reversal? / Why is "largest absolute segment delta" such a dangerous default heuristic?

Metric Design

Q45. Walk me through numerator/denominator discipline in defining a rate metric.

Ideal answer. Every rate is a numerator over a denominator, and most metric bugs are denominator bugs — the numerator is usually obvious, the denominator is where definitions silently diverge. For "checkout conversion rate," is the denominator sessions that reached checkout, all sessions, or users? Each gives a different number and answers a different question. My discipline: state the unit of analysis explicitly (session vs user vs event), ensure the numerator is a strict subset of the denominator population (you can't have purchases from sessions that aren't in the denominator), align the time windows of both, and define inclusion/exclusion rules (bots, internal traffic, refunds) once. A rate where numerator and denominator are computed over mismatched populations or windows is the single most common way two teams "measure the same metric" and get different answers. **Why Helios demonstrates it.** Helios's `reached_*` monotonic max-downstream flags in `int_ga4__funnel_steps` guarantee numerator/denominator coherence — a session that reached purchase is provably counted in every upstream step's denominator — so step rates are arithmetically consistent and reconcile to $\leq 0.5\%$. **Follow-ups.** Give a case where changing only the denominator flips the conclusion. / How do you align numerator and denominator time windows for a lagging conversion? / Why is the unit of analysis (session vs user) a definitional choice, not a detail?

Q46. Why are ratio metrics dangerous, and how does Simpson's paradox show up in them?

Ideal answer. Ratio metrics (rates, averages-of-ratios, per-user values) are dangerous because they don't aggregate the way people assume — you cannot average two groups' rates to get the pooled rate unless you weight by denominator, and a weighted aggregate can move opposite to every one of its parts. That's Simpson's paradox: each segment's conversion rate can *rise* while the overall rate *falls*, purely because traffic mix shifted toward a lower-converting segment. The defense is to always decompose an aggregate-rate change into mix vs rate effects before interpreting it, and to be explicit about the weighting. Operationally I also avoid "average of per-user ratios" when I mean "ratio of totals" — they're different metrics and the per-user-ratio version is heavily distorted by low-denominator users. **Why Helios demonstrates it.** Simpson's paradox is the exact pathology Helios was built to dissolve: `decompose_change` mechanically separates the mix component ($\sum dw_i r_i$) from the rate component ($\sum w_i dr_i$), so an aggregate that fell while segments rose is correctly attributed to composition rather than mistaken for behavioral

decline. **Follow-ups.** Show me a numeric example where pooled rate moves opposite to every segment. / When is "ratio of sums" right and "average of ratios" wrong? / How does denominator weighting change which aggregate you should report?

Q47. What does it mean for a metric to be "governed," and why does it matter?

Ideal answer. A governed metric has a single, version-controlled, centrally-owned definition — its SQL logic, unit, filters, and dimensions live in one registry, and every consumer (dashboard, model, agent, report) derives from that definition rather than re-implementing it. It matters because the alternative is *metric drift*: ten teams write ten slightly different "active user" queries, three numbers reach the exec, and the meeting becomes a debate about whose SQL is right instead of what to do. Governance buys consistency, auditability (you can trace any number to its definition), and trust. The tradeoff is upfront cost and some rigidity — you can't redefine a metric on a whim — but for any metric that drives decisions, that rigidity is the point. The hard part of analytics isn't computing a number; it's everyone computing the *same* number. **Why Helios demonstrates it.** Helios's semantic layer (models/semantic/semantic_models.yml: 28 metrics + 16 dimensions, referential-integrity compiled, mapping 1:1 to dbt MetricFlow) is the single source of truth, and semantic-mcp's `build_query` is the *only* path to SQL — so there are zero hallucinated columns and no agent can author an off-definition metric. **Follow-ups.** How do you stop teams from re-implementing a metric outside the registry? / What's the cost of governance and when is it not worth it? / How do you migrate a legacy ad-hoc metric into a governed definition?

Q48. How do you version and evolve a metric definition without breaking trust in historical numbers?

Ideal answer. Metric definitions *must* change (new business logic, fixed bugs, new exclusions), and the danger is silently changing a definition so that a chart's history shifts underneath everyone — last quarter's number no longer matches last quarter's deck. I handle it like code: definitions are version-controlled with semantic versioning, changes go through review with a documented rationale, and a breaking redefinition gets a *new* metric version (`purchase_rate_v2`) rather than mutating v1 in place. I keep a changelog mapping when each version was active, and where possible I backfill both versions during a transition so dashboards can show the discontinuity explicitly rather than hide it. The principle: a metric's history should be reproducible, and any redefinition should be announced and auditable, never silent. **Why Helios demonstrates it.** Because Helios's metrics live in a git-versioned `semantic_models.yml` under CI, any definition change is a reviewable diff gated by the offline benchmark — the same PR that changes a metric must keep decomposition $\text{MAPE} \leq 10\%$ and 0 hallucinated columns, so a redefinition can't silently regress historical correctness. **Follow-ups.** When do you version-bump a metric vs edit it in place? / How do you communicate a metric redefinition to dashboard consumers? / How would you reproduce a number computed under an old metric version?

Q49. Design a metric for "checkout health" from scratch — talk me through your reasoning.

Ideal answer. I'd start from the decision the metric serves: is checkout *working*, and where does it leak? A single headline rate (begin_checkout → purchase) is the lagging summary, but health needs the step-level breakdown — add_shipping_info, add_payment_info, purchase — because "checkout is broken" is meaningless without the leaking step. So I'd define checkout health as the conversion rate of each downstream step with the *same* denominator discipline (sessions that reached the prior step), surfaced alongside a guardrail (payment-error rate) and segmented by device and channel since checkout friction is wildly device-dependent. I'd explicitly choose session-grain over user-grain given the data, state the time window, and exclude refunds from revenue. The output isn't one number — it's a small governed metric set that localizes the leak and reconciles to the funnel. **Why Helios demonstrates it.** This mirrors Helios's funnel marts exactly: begin_checkout → add_shipping_info → add_payment_info → purchase as monotonic step flags in int_ga4_funnel_steps, each step a governed metric in the semantic layer with consistent denominators, so "checkout health" is decomposable to the exact leaking step and segment. **Follow-ups.** Why a step-level set rather than one composite checkout score? / How do you pick the denominator for each checkout step? / Which guardrail would you pair with checkout conversion and why?

Q50. How do you handle metric definitions that must reconcile across many consumers and stay correct over time?

Ideal answer. Reconciliation is an engineering problem, not a willpower problem: I make one definition the source of truth, generate all downstream SQL from it, and add automated checks that the same metric computed two ways agrees within a tolerance. Concretely that means a semantic registry, dry-run cost gates so a query can't silently scan the wrong (more expensive, differently-scoped) tables, and reconciliation tests in CI that fail the build if a metric drifts beyond, say, 0.5%. I also pin the *byte/scope* of each query so "the same metric" can't accidentally read a different partition. The mature recognition is that correctness isn't a one-time verification — it's a continuously-enforced invariant, because data, definitions, and consumers all change, and the only durable defense is automation that catches drift on every change. **Why Helios demonstrates it.** Helios enforces exactly this: warehouse-mcp's mandatory dry_run → run_query byte-budget gate with reconcile-to-≤0.5%, the governed semantic layer as the single SQL source, and CI gates that block any PR regressing the offline benchmark — so metric correctness is a continuously-enforced invariant, not a hope. **Follow-ups.** What tolerance do you set for reconciliation and why? / How does a cost/byte gate also protect metric correctness? / What CI checks would you add to catch metric drift before it ships?